1. Definir la relación `invertir(?L1, ?L2)` que se verifique si L2 es la lista obtenida invirtiendo el orden de los elementos de la lista L1 (sin utilizar el predicado predefinido `reverse/2`).

2. Definir la relación `aplanac(+ListaAnidada, -ListaPlana)` que reciba una lista con sublistas anidadas y devuelva todos sus elementos en una lista plana de un solo nivel. Por ejemplo:

```
?- aplana([a, [b, [c, d]], e], L).
L = [a, b, c, d, e]
```

3. Definir la relación `factorial(+X, ?Y)` que se verifique si Y es el factorial del número entero no negativo X. Por ejemplo:

```
?- factorial(3, X).
X = 6
```

4. Una misma información — quién corre en cada escudería, con qué motor y de qué nacionalidad — se puede representar de más de una manera en Prolog. Veamos dos estructuras para la parrilla de F1 2026.

   1 (anidada):

```
escuderia(mclaren, mercedes, [piloto(norris,britanico),
↪  piloto(piastri,australiano)]).
escuderia(mercedes, mercedes, [piloto(russell,britanico),
↪  piloto(antonelli,italiano)]).
escuderia(redbull, redbull, [piloto(verstappen,neerlandes),
↪  piloto(hadjar,frances)]).
escuderia(ferrari, ferrari, [piloto(hamilton,britanico),
↪  piloto(leclerc,monegasco)]).
escuderia(astonmartin, honda, [piloto(alonso,espanol),
↪  piloto(stroll,canadiense)]).
escuderia(williams, mercedes, [piloto(alon,tailandes),
↪  piloto(sainz,espanol)]).
escuderia(audi, audi, [piloto(hulkenberg,aleman),
↪  piloto(bortoleto,brasileno)]).
escuderia(haas, ferrari, [piloto(ocon,frances), piloto(bearman,britanico)]).
escuderia(alpine, mercedes, [piloto(colapinto,argentino),
↪  piloto(gasly,frances)]).
escuderia(cadillac, ferrari, [piloto(perez,mexicano),
↪  piloto(bottas,finlandes)]).
escuderia(racingbulls, redbull, [piloto(lawson,neozelandes),
↪  piloto(lindblad,britanico)]).
```

   2 (plana):

```
piloto(norris, mclaren, mercedes, britanico).
piloto(piastri, mclaren, mercedes, australiano).
piloto(russell, mercedes, mercedes, britanico).
piloto(antonelli, mercedes, mercedes, italiano).
piloto(verstappen, redbull, redbull, neerlandes).
piloto(hadjar, redbull, redbull, frances).
piloto(hamilton, ferrari, ferrari, britanico).
piloto(leclerc, ferrari, ferrari, monegasco).
piloto(alonso, astonmartin, honda, espanol).
```

```
piloto(stroll, astonmartin, honda, canadiense).
piloto(alon, williams, mercedes, tailandes).
piloto(sainz, williams, mercedes, espanol).
piloto(hulkenberg, audi, audi, aleman).
piloto(bortoleto, audi, audi, brasileno).
piloto(ocon, haas, ferrari, frances).
piloto(bearman, haas, ferrari, britanico).
piloto(colapinto, alpine, mercedes, argentino).
piloto(gasly, alpine, mercedes, frances).
piloto(perez, cadillac, ferrari, mexicano).
piloto(bottas, cadillac, ferrari, finlandes).
piloto(lawson, racingbulls, redbull, neozelandes).
piloto(lindblad, racingbulls, redbull, britanico).
```

(a) Discutir con los compañeros las ventajas y desventajas de cada estructura (¿cuál es más fácil de consultar si quiero el motor de un piloto? ¿y si quiero saber quiénes son compañeros de equipo? ¿y si mañana ficha un piloto nuevo? etc). Resumir las conclusiones como se muestra en el Cuadro 1.

(b) Resolver, usando la Estructura 2 (`piloto/4`):

I. ¿Qué piloto argentino corre en la Fórmula 1 2026 y en qué escudería?

II. ¿Cuántas escuderías distintas usan motor Mercedes? ¿Y motor Ferrari?

III. ¿Todos los pilotos británicos y la escudería en la que corre cada uno?

IV. Definir `compañero_de_equipo(?X, ?Y)`: X e Y corren para la misma escudería (`X \= Y`).

V. Definir `mismo_motor(?X, ?Y)`: los autos de X e Y llevan el mismo motor, aunque corran para escuderías distintas (`X \= Y`).

VI. Definir `mismo_pais(?X, ?Y)`: X e Y son pilotos de la misma nacionalidad (`X \= Y`). Preguntar por todos los pares de pilotos británicos.

VII. Definir `esPiloto(X)`: X es un piloto existente en la base de datos.

VIII. Preguntar por los nombres y escuderías de todos los pilotos de la base de datos.

IX. Definir `motor_de(?Piloto, ?Motor)`: Motor es el motor que usa Piloto, a partir de su escudería.

X. Definir `cantidad_por_motor(+ListaPilotos, +Motor, ?N)`: si `ListaPilotos` es una lista de nombres de pilotos, N es la cantidad de ellos que corre con el motor Motor.

Aspecto	**Estructura anidada**	**Estructura plana**
Buscar el motor de UN piloto		
Listar compañeros de equipo		
Agregar un piloto nuevo		
Legibilidad para alguien que no programó la base		

Cuadro 1: Comparación de estructuras

5. Un rectángulo se divide en 4 rectángulos de base y altura entera. El problema consiste en determinar la superficie del subrectángulo superior derecho a partir de las superficies de los subrectángulos inferior izquierdo, inferior derecho y superior izquierdo. Definir la relación

`rectángulo(+S1,+S2,+S3,-S4)` que se verifique si S1 es la superficie del subrectángulo inferior izquierdo, S2 la del inferior derecho, S3 la del superior izquierdo y S4 la del superior derecho. Por ejemplo,

```
?- rectangulo(6,10,21,S).
S = 35
?- rectangulo(10,6,35,S).
S = 21
?- rectangulo(6,10,35,S).
No
```

Tips: Utilice el subrectángulo de mayor superficie inicialmente.
Puede usar el predicado `between(+Low, +High, ?Value)` que verifica si un entero cae en un rango específico, o que genera por backtracking los enteros en un rango.

6. Determinar si un triángulo es **equilátero**, **isósceles** o **escaleno**.

   - Un triángulo **equilátero** tiene sus tres lados de la misma longitud.
   - Un triángulo **isósceles** tiene al menos dos lados de la misma longitud. (A veces se especifica como que tiene exactamente dos lados iguales, pero para los propósitos de este ejercicio diremos que tiene al menos dos).
   - Un triángulo **escaleno** tiene todos sus lados de longitudes diferentes.

   Nota: Para que una figura sea un triángulo, todos los lados deben tener longitud > 0, y la suma de las longitudes de dos lados cualesquiera debe ser mayor o igual que la longitud del tercer lado.

   Los *triángulos degenerados* son aquellos en los que la suma de dos lados es **igual** a la longitud del tercer lado, por ejemplo, $1, 1, 2$. En este ejercicio no se incluyen pruebas para triángulos degenerados. Puedes manejar esas situaciones si lo deseas, o ignorarlas con seguridad.

   En ecuaciones: sean a, b y c los lados del triángulo. Entonces las tres expresiones siguientes deben ser verdaderas:

   $$a + b \geq c, \qquad b + c \geq a, \qquad a + c \geq b.$$

   Se debe implementar el predicado `triangle/4` con la siguiente signatura:

```
triangle(+Lado1, +Lado2, +Lado3, ?Tipo)
```

   Donde `Tipo` es una cadena de texto que puede ser `equilátero`, `isósceles` o `escaleno`. Por ejemplo:

```
?- triangle(2, 2, 2, "equilateral").
true.

?- triangle(2, 2, 2, Tipo).
Tipo = "equilateral".

?- triangle(3, 4, 4, "isosceles").
true.

?- triangle(5, 4, 6, "scalene").
true.
```

```
?- triangle(1, 1, 3, "isosceles").
false.

?- triangle(1, 1, 3, Tipo).
false.
```

7. El objetivo es colocar ocho reinas en un tablero de ajedrez de modo que ninguna reina ataque a otra; es decir, que no haya dos reinas en la misma fila, la misma columna o la misma diagonal. Generalizamos este problema original permitiendo una dimensión arbitraria $N \times N$ del tablero de ajedrez.

   Tip: podemos representar las posiciones de las reinas como una lista de números $1..N$. Por ejemplo, $[4, 2, 7, 3, 6, 8, 5, 1]$ significa que la reina en la primera columna está en la fila 4, la reina en la segunda columna está en la fila 2, etc.

   Se debe implementar un predicado `queens(+N, -Qs)`. Donde `N` es la dimensión del tablero y `Qs` es la lista que representa una solución válida (una permutación de $1..N$ que satisface las restricciones). Por ejemplo:

```
?- queens(8, Qs).
Qs = [4,2,7,3,6,8,5,1] ;
Qs = [5,2,8,1,3,7,4,6] ;
...

?- queens(4, Qs).
Qs = [2,4,1,3] ;
Qs = [3,1,4,2].
```

8. Dada una lista de números enteros, encontrar una forma correcta de insertar signos aritméticos de modo que el resultado sea una ecuación correcta. Por ejemplo, con la lista de números $[2, 3, 5, 7, 11]$ podemos formar las ecuaciones $2-3+5+7 = 11$ o $2 = (3*5+7)/11$ (y más). Se debe implementar el predicado `equation/3`. Tip: usar predicados auxiliares `term/2`, `binterm/3` y `split/3`. Por ejemplo:

```
?- equation([2,3,5,7,11],L,R).
L = 2
R = 3-(5+(7-11)) ;
L = 2,
R = (3*5+7)/11 ;
L = 2*(3-5),
R = 7-11 ;
...
```